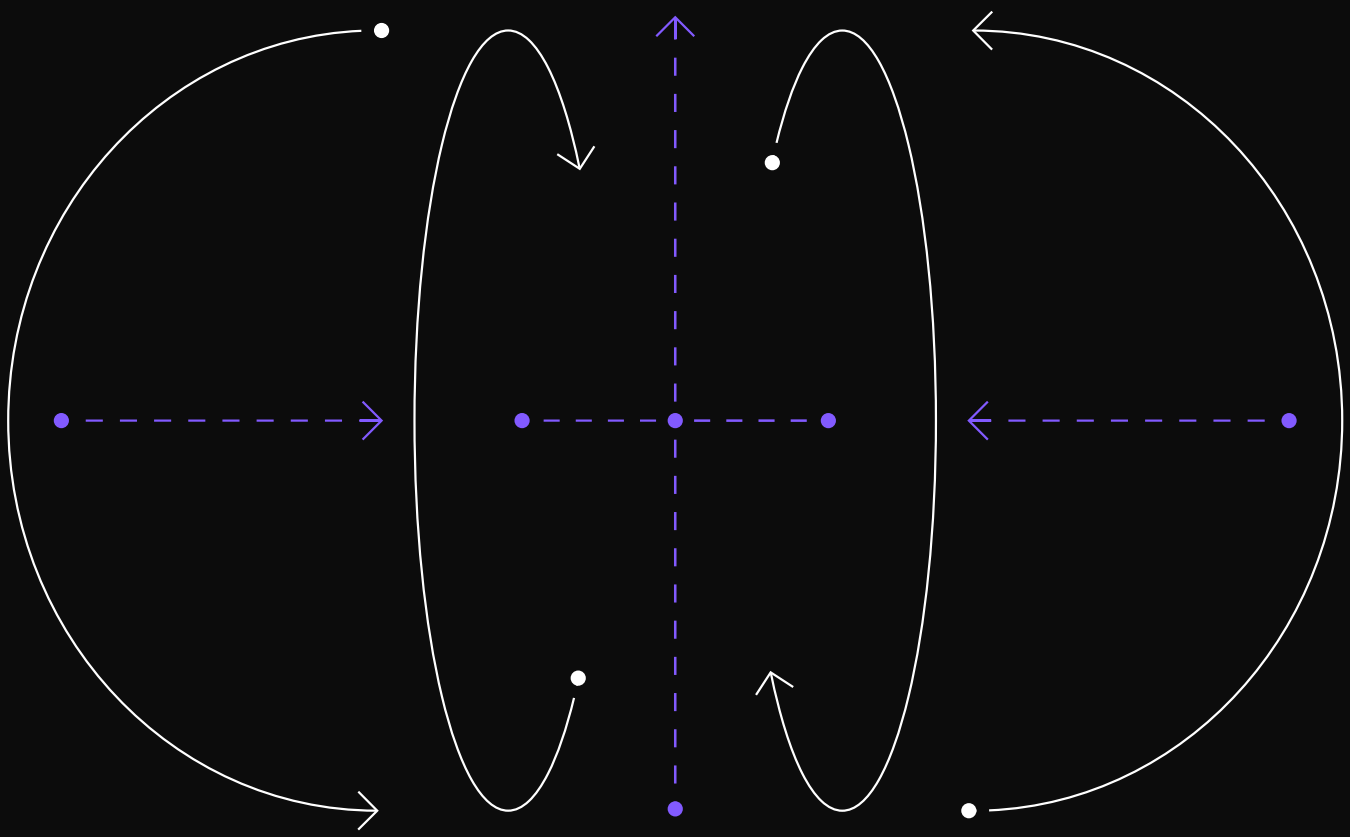
Машинное обучение

Лонгрид

Неделя 3

Метрические алгоритмы



Содержание

1. **Что такое метрические алгоритмы** 4
   1.1. Распространённые метрики 4
2. **Ближайший центроид** 8
   2.1. Плюсы и минусы метода 10
3. **Алгоритм k ближайших соседей (kNN)** 11
   3.1. Алгоритм kNN для задач классификации и регрессии 12
   3.2. Как выбрать значение k 15
   3.3. Взвешенный kNN 16
4. **Плюсы и минусы метода** 19
   4.1. Проклятие размерности 20
5. **Регрессия Надарая–Ватсона** 21
   5.1. Плюсы и минусы метода 23
6. **Fast kNN** 23
7. **Если кратко** 26

Метрические алгоритмы

Контекст

Задача машинного обучения — подобрать модель и её параметры так, чтобы приближённо описать сложную функциональную зависимость между признаками и целевой переменной. Но если зависимость слишком сложная для параметрического описания, можно использовать принципиально иной подход.

Применение

В отличие от методов, строящих сложную обобщающую модель, метрические алгоритмы работают напрямую с данными, опираясь на меру их схожести. Эта простота не мешает им эффективно решать задачи в таких областях, как медицинская диагностика, банковская сфера, обработка текстов и классификация изображений.

Краткий план

- → Что такое метрические алгоритмы
- → Ближайший центроид
- → Алгоритм k ближайших соседей (kNN)
- → Плюсы и минусы метода
- → Регрессия Надарая–Ватсона
- → Fast kNN
- → Если кратко

1. Что такое метрические алгоритмы

Чтобы узнать о новом объекте, можно сравнить его с уже известными объектами.

ПРИМЕР

Чтобы определить диагноз пациента, можно сопоставить его симптомы и анализы с другими пациентами. Если показатели похожи, то, вероятно, это одна и та же болезнь.

В этом примере вместо того, чтобы искать некоторую глобальную формулу, мы фокусируемся на поиске локальных взаимосвязей — близости между объектами в пространстве признаков.

Так и работают метрические алгоритмы. По-другому их называют *distance-based*, *memory-based*, *instance-based*, *lazy learning*.

ОПРЕДЕЛЕНИЕ

Метрические алгоритмы — методы машинного обучения, которые делают предсказание для нового объекта, сравнивая его с соседями из обучающей выборки.

Расстояние, или **метрика**, измеряет близость между объектами в пространстве признаков.

Чтобы функция $d(x, y)$ могла называться функцией расстояния, она должна отвечать следующим условиям.

1. **Неотрицательность:** $d(x, y) \geq 0$ для всех x, y из нашего пространства.
   Расстояние не может быть отрицательным.
2. **Тождество неразличимых:** $d(x, y) = 0$ тогда и только тогда, когда $x = y$.
   Если сравнить одну и ту же точку, расстояние будет равно нулю.
3. **Симметричность:** $d(x, y) = d(y, x)$.
   Расстояние от x до y равно расстоянию от y до x, то есть неважно, откуда начинать считать.
4. **Неравенство треугольника:** $d(x, y) \leq d(x, z) + d(z, y)$.
   Добавление промежуточной точки от x к y не может уменьшить расстояние: прямой путь всегда самый короткий.

1.1. РАСПРОСТРАНЁННЫЕ МЕТРИКИ

Существует много способов измерить расстояние, но выбор подходящей метрики основывается на типе данных и на том, что в текущем контексте подразумевается под схожестью. Кроме того, зачастую необходимы масштабирование или стандартизация.

Рассмотрим самые популярные метрики.

1. **Евклидово расстояние** (*Euclidean distance*, L_2-норма).
   Длина кратчайшего пути между двумя точками в пространстве, то есть длина отрезка на прямой, соединяющей x и y. Это самая интуитивная и часто используемая метрика. Она очень чувствительна к масштабу признаков.

$$d_2(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}.$$

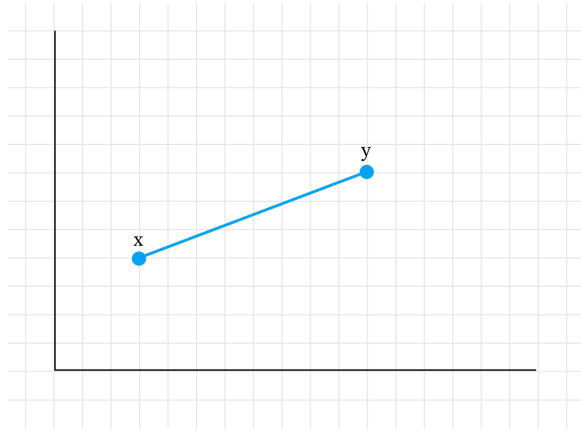


Рис. 1. Геометрическая интерпретация евклидова расстояния в двумерном пространстве

ПРИМЕР

Если нужно оценить кредитный рейтинг клиента банка, признак «возраст» будет исчисляться в десятках, а признак «доход» — в тысячах. Тогда второй признак будет составлять бóльшую часть рассчитанного расстояния, в то время как первый практически не учтётся, хотя может быть критически важен.

Поэтому перед использованием евклидова расстояния необходимо масштабировать или стандартизировать данные (например, с помощью StandardScaler или MinMaxScaler из sklearn). К тому же квадрат разностей усиливает влияние выбросов: если один клиент зарабатывает в 10 раз больше, чем остальные, квадрат такой разности будет очень большим числом.

2. **Манхэттенское расстояние** (*Manhattan distance*, L_1-норма), или **расстояние городских кварталов** (*city block distance*).
   Сумма длин катетов, а не гипотенуза. Основная идея: невозможно пройти от одной аудитории в другую «напрямую» — нужно двигаться по коридорам (по аналогии с улицами и кварталами Манхэттена).

$$d_1(x, y) = \sum_{i=1}^{n}|x_i - y_i|.$$

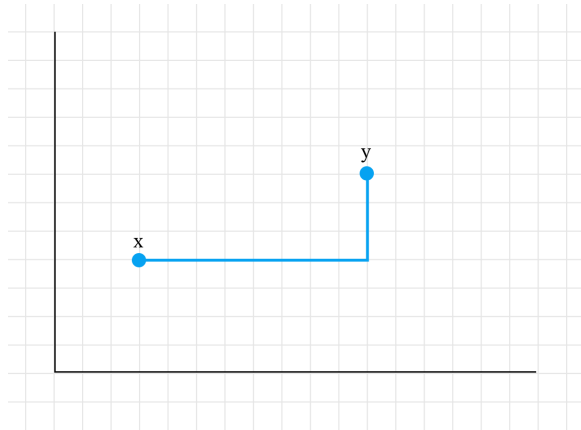


Рис. 2. Геометрическая интерпретация манхэттенского расстояния в двумерном пространстве

Как и евклидово, это расстояние достаточно чувствительно к масштабу признаков: рекомендуются масштабирование и стандартизация. Однако оно более устойчиво к отдельным большим выбросам по одному признаку благодаря отсутствию возведения в квадрат и иногда лучше работает в пространствах с большой размерностью.

3. **Расстояние Минковского** (*Minkowski distance*, L_p-норма).
Общий случай евклидова и манхэттенского расстояний. Позволяет гибко настраивать метрику, но чаще всего используются L_1- и L_2-нормы. Требует масштабирования данных.

$$d_p(x,\ y) = \left(\sum_{i=1}^{n} \|x_i - y_i\|^p\right)^{\frac{1}{p}}.$$

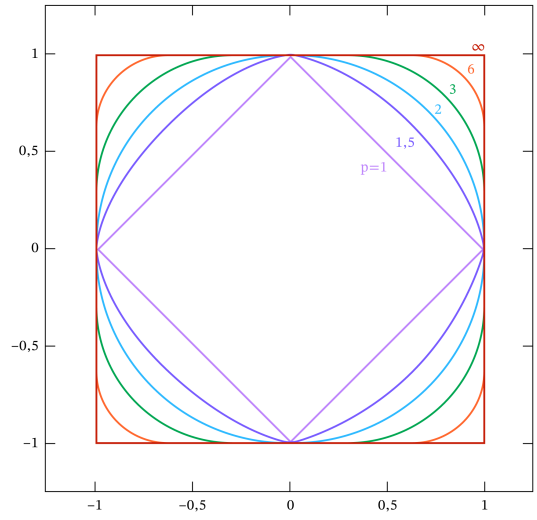


Рис. 3. Единичные окружности для различных p-норм (метрик Минковского). Источник

Параметр $p \geq 1$ определяет порядок нормы:

- $p = 1$ — манхэттенское;
- $p = 2$ — евклидово;
- $p \to \infty$ — **расстояние Чебышёва** (или **расстояние шахматной доски**): оно показывает максимальное расстояние по любой из координат. Эта метрика используется редко, в частности она применяется в компьютерном зрении при анализе квадратной окрестности пикселя.

ПРИМЕР

В трёхмерном пространстве расстояние Чебышёва между точками $(x_1,\ x_2,\ x_3)$ и $(y_1,\ y_2,\ y_3)$ равно

$$\max\left(|x_1 - y_1|,\ |x_2 - y_2|,\ |x_3 - y_3|\right).$$

4. **Косинусное расстояние** (*cosine distance*).
Угловое сходство между двумя векторами. Чем меньше угол, тем более векторы однонаправлены и, соответственно, «похожи» вне зависимости от их длины (магнитуды).

$$d_{\cos}(x,\ y) = 1 - \cos(\theta).$$

Здесь $\cos(\theta)$ — косинусное сходство:

$$\cos(\theta) = \frac{\langle x,\ y\rangle}{\|x\|\ \|y\|} = \frac{\sum\limits_{i=1}^{n} x_i y_i}{\sqrt{\sum\limits_{i=1}^{n} x_i^2} \cdot \sqrt{\sum\limits_{i=1}^{n} y_i^2}}.$$

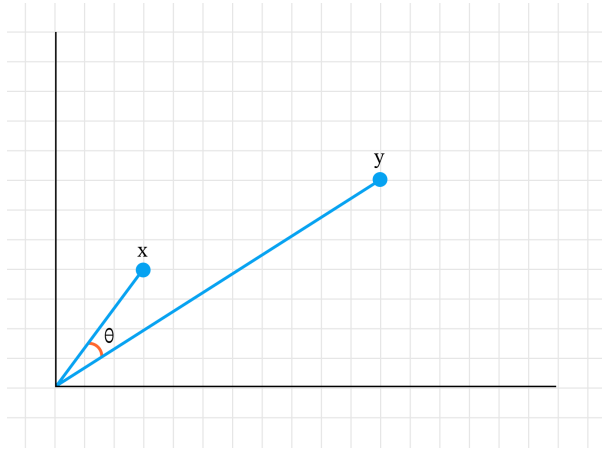


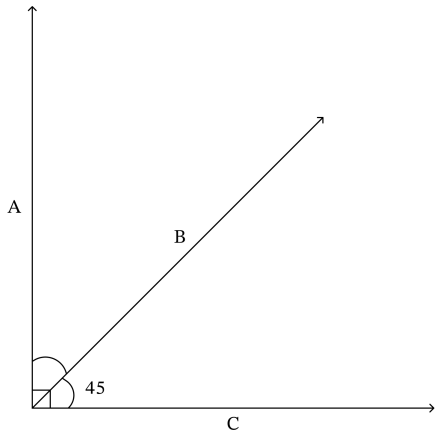
Рис. 4. Геометрическая интерпретация косинусного расстояния в двумерном пространстве

Оно варьируется от 1 до -1, где:

- 1 — угол $0°$ (максимальное сходство);
- 0 — угол $90°$ (ортогональность). Если сравнить центрированные относительно средних значений векторы $x' = x - \text{mean}(x)$, $y' = y - \text{mean}(y)$, то можно сделать вывод об их некоррелированности;
- -1 — угол $180°$ (противоположность).

Косинусное расстояние — это не метрика в строгом смысле, потому что нарушает две аксиомы:

1) тождество неразличимых ($d_{\cos}(x, y) = 0$ не гарантирует $x = y$, а только говорит об их коллинеарности);
2) неравенство треугольника (функция $f(\theta) = 1 - \cos(\theta)$ растёт нелинейно: медленнее для малых углов и быстрее для больших, поэтому сумма расстояний двух маленьких шагов оказывается меньше, чем расстояние одного большого шага).



$$\angle AC = 90^\circ$$
$$\angle AB = 45^\circ$$
$$\angle BC = 45^\circ$$

$$1 - cos(AC) = 1 - \cos 90 = 1$$
$$(1 - \cos(AB)) + (1 - \cos(BC)) =$$
$$= (1 - cos(45)) \times 2 = 2 - \sqrt{2} \approx 0,6$$
$$0,6 < 1, \text{ а должно быть } \geq$$

Рис. 5. Неравенство треугольника

Тем не менее оно популярно для анализа текстовых данных. Когда тексты представляются в виде векторов, их направление отражает тематическое распределение слов. В этом случае косинусное расстояние позволяет определить документы со схожей тематикой.

5. **Расстояние Хэмминга** (*Hamming distance*).
Метрика, которая сравнивает два вектора (слова, списки) одинаковой длины и считает количество различающихся соответственных символов. Расстояние Хэмминга можно использовать для сравнения категориальных признаков, но для числовых оно не подойдёт.

$$d_H(x, y) = \sum_{i=1}^{n} I(x_i \neq y_i),$$

где $I(\cdot)$ — индикаторная функция. Она возвращает 1, если условие истинно, и 0, если ложно.
В таблице можно посмотреть пример, как считать расстояние Хэмминга.

Позиция i	**1**	**2**	**3**	**4**	**5**
x_i	0	1	1	0	1
y_i	1	1	0	0	0
$x_i \neq y_i$	Да	Нет	Да	Нет	Да

Подсчитываем количество различающихся символов (слов «Да» в последней строке), получаем ответ 3. Значит, итоговое расстояние равно 3.

2. Ближайший центроид

Вышеуказанные метрики лежат в основе алгоритмов, которые классифицируют объекты по степени их похожести. Один из самых базовых и наглядных — метод ближайшего центроида.

ОПРЕДЕЛЕНИЕ

Метод ближайшего центроида — это базовый метрический алгоритм, который сравнивает новый объект с другим, «типичным» для конкретной группы.

ПРИМЕР

Ребёнок учится различать животных с помощью картинок в энциклопедии. Увидев кошку на улице, он мысленно сравнит её с изображением из книги: небольшое животное с четырьмя лапами, длинным хвостом, округлой мордой, треугольными ушами (и далее по списку признаков).

Сравниваемый объект относится к тому классу, центроид (центр) которого ближе всего к нему в признаковом пространстве.

ОПРЕДЕЛЕНИЕ

Центроид — это вектор, координаты которого соответствуют средним значениям признаков всех объектов обучающей выборки, принадлежащих отдельному классу.

ПРИМЕР

Обучающая выборка состоит из n элементов: $\{(x_1, y_1), \dots, (x_n, y_n)\}$, где:

- x_i — вектор признаков;
- y_i — метка класса $\in K = \{1, \dots, k\}$.

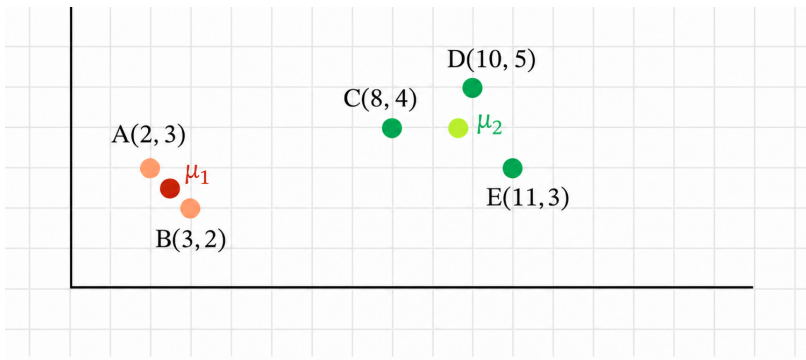
На этапе обучения вычислим центроиды:

$$\mu_k = \frac{1}{|I_k|} \sum_{i \in I_k} x_i,$$

где:

- $I_k = \{i \mid y_i = k\}$ — список индексов объектов, принадлежащих классу k;
- $|I_k|$ — их количество.

В результате получился список центроидов $\{\mu_1, \dots, \mu_k\}$. Хранить обучающую выборку в памяти больше не нужно — только эти k центроидов.



$$\mu_1 = \left(\frac{2+3}{2}, \frac{3+2}{2} \right) = (2{,}5,\ 2{,}5)$$

$$\mu_2 = \left(\frac{8+10+11}{3}, \frac{4+5+3}{3} \right) = (9{,}67,\ 4)$$

Рис. 6. Пример вычисления центроидов

На этапе предсказания вычисляем расстояние от нового элемента $\vec{x}$ до каждого из k центроидов, используя некоторую метрику (чаще всего евклидово расстояние), и находим ближайший класс $\hat{y}$:

$$\hat{y} = \arg\min_{\ell \in K} d(\vec{x}, \mu_\ell).$$

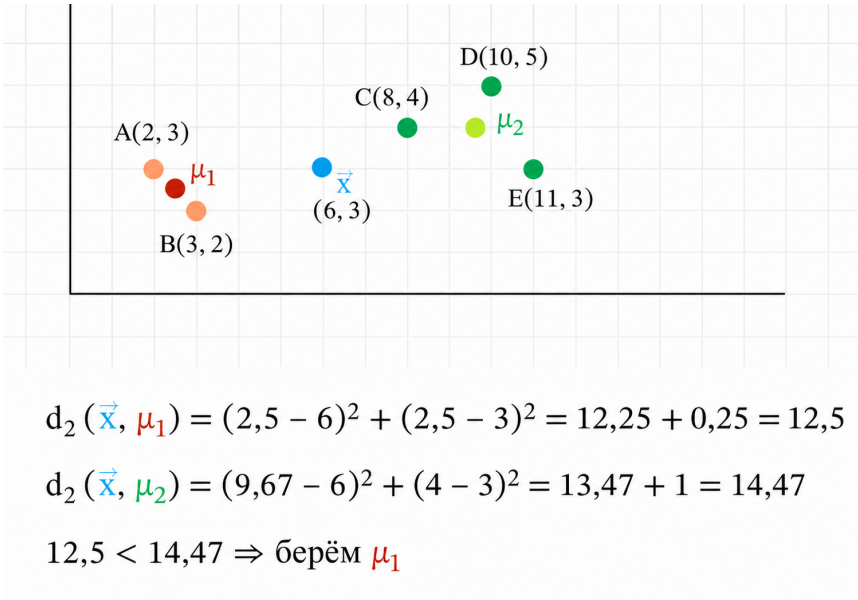


Рис. 7. Классификация нового объекта методом ближайшего центроида

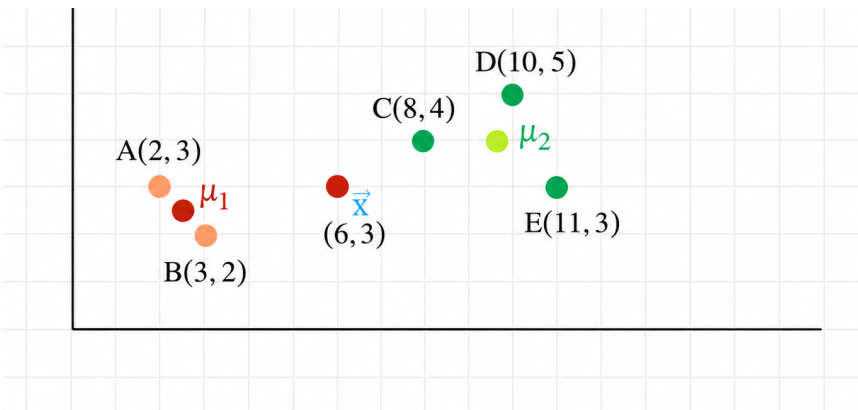


Рис. 8. Классификация нового объекта методом ближайшего центроида

После добавления нового объекта в класс центроид можно обновить. Такое обновление имеет смысл в онлайн-обучении, где данные поступают постепенно. Но обычно векторы вычисляются один раз на изначальной обучающей выборке.

2.1. ПЛЮСЫ И МИНУСЫ МЕТОДА

Плюсы

- Метод легко понять и реализовать с нуля: мы считаем каждый центроид класса (векторы усреднённых признаков всех объектов класса) и расстояния до них.
- Он очень быстрый: на этапе предсказания нужно сравнить объект с k центроидами, а не со всеми n точками.
- Не нужно хранить всю обучающую выборку, только центроиды: так экономится много ресурсов. Если мы хотим обновлять центроиды по мере поступления новых объектов, нужно хранить сумму векторов и количество точек: благодаря этому можно не пересчитывать всё сначала, а выполнить два сложения и одно деление.

Минусы

- Может разделить только линейно разделимые классы и не справится с извилистыми границами решений.

  ПРИМЕР

  При использовании евклидова расстояния границей между любой парой классов станет линейная гиперплоскость, перпендикулярная отрезку, соединяющему центроиды этих пар классов. Такая структура называется **диаграммой Вороного**.

- В некоторых случаях плохо работает со сложными структурами и кластерами данных.

  ПРИМЕР

  В качестве усреднения центроид будет плохим представителем класса:

  - в одном классе точки компактно сгруппированы, а в другом — распределены в более «широкой» области;
  - классы имеют невыпуклую форму (например, полумесяц или S-образная кривая).

- Чувствителен к выбросам из-за расчёта среднего значения: может сильно сдвинуть центроид.

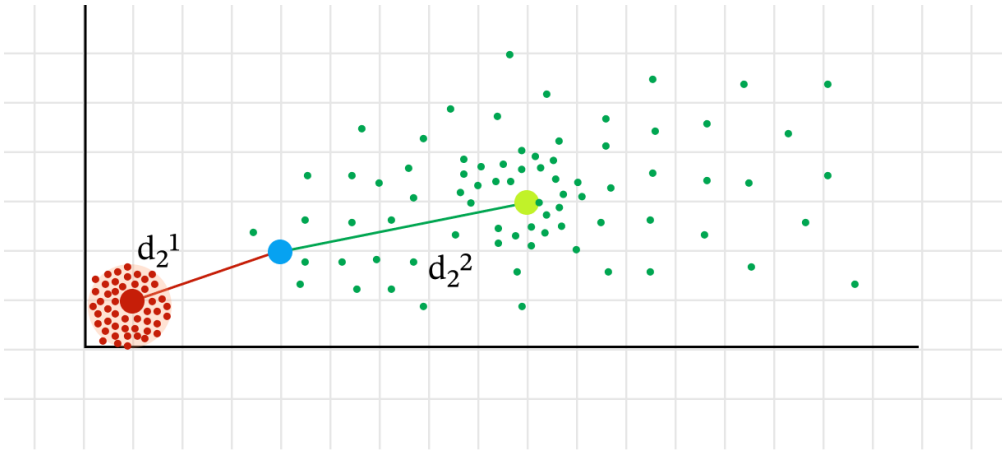


Рис. 9. Пример неверной классификации из-за формы кластера

3. Алгоритм k ближайших соседей (kNN)

Метод ближайшего центроида прост, но у него есть ограничение: он учитывает только «усреднённый» образ класса, игнорируя индивидуальные особенности объектов. Эволюция этой идеи — вместо одного усреднённого представителя класса:

- смотреть на непосредственных соседей в некоторой окрестности новой точки;
- делать предсказание, основываясь на близости к окружающим объектам.

Существует два основных способа определения окрестности $N(x)$ вокруг точки x.

1. По фиксированному радиусу r: в нём рассматриваются все точки, попавшие в область шара. Но есть сложности, например может возникнуть вопрос определения r: как быть, если в фиксированном радиусе нет ни одной точки?
2. По фиксированному количеству соседей k: с ними сравнивается новая точка.

Именно второй подход лежит в основе одного из самых известных и интуитивно понятных метрических методов — алгоритма k ближайших соседей.

ОПРЕДЕЛЕНИЕ

Алгоритм k ближайших соседей (англ. *k-nearest neighbors*, или *kNN*) — это метрический метод, который предсказывает свойства нового объекта на основе k наиболее близких к нему объектов из обучающей выборки.

ПРИМЕР

Предсказание рейтинга фильма

Задача: предсказать, какую оценку пользователь поставит фильму.

РЕШЕНИЕ

1. Находим k пользователей, наиболее похожих на данного.
2. Смотрим, какие оценки они поставили этому фильму.
3. Усредняем их оценки.

Получается простая, но мощная модель рекомендаций.

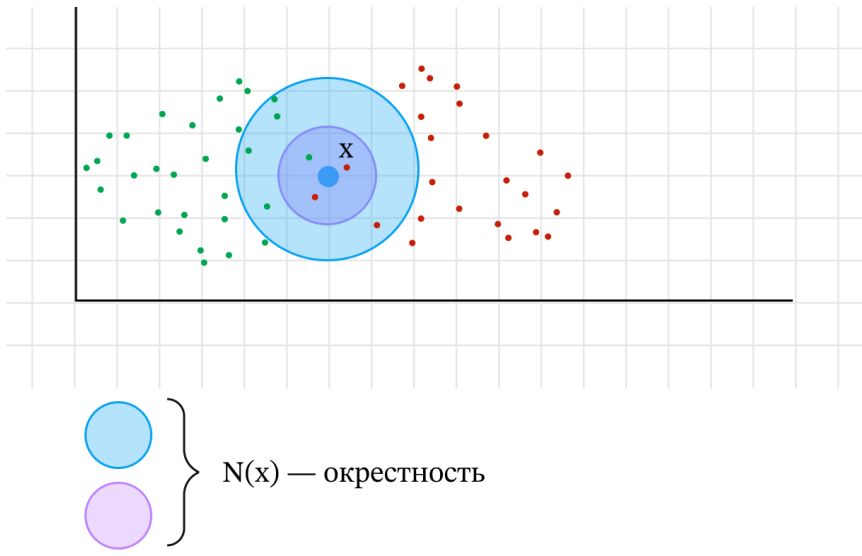


Рис. 10. Окрестность вокруг точки

kNN — непараметрический метод, в котором для нового объекта определяются ближайшие точки из обучающей выборки, а предсказание строится на основе свойств этих соседей. Алгоритм применяется в задачах классификации и регрессии.

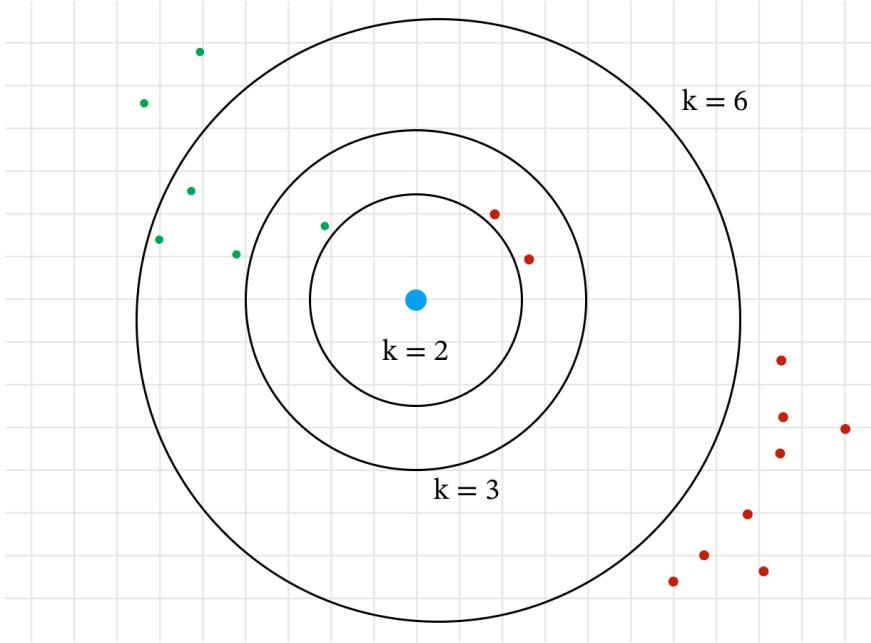


Рис. 11. Окрестность вокруг точки, основанная на количестве соседей

3.1. АЛГОРИТМ KNN ДЛЯ ЗАДАЧ КЛАССИФИКАЦИИ И РЕГРЕССИИ

Обучающая выборка состоит из объектов $(x_1, y_1), \ldots, (x_n, y_n)$, где x_i — вектор признаков, y_i — целевая переменная (метка класса или числовое значение). Для нового объекта $\vec{x}$ алгоритм kNN ищет k наиболее похожих объектов в обучающей выборке и выносит предсказание на основе их целевых значений.

Алгоритм работает следующим образом.

1. Подбираем количество соседей, на которое будем ориентироваться. Это будет целое положительное число $1 \leq k \leq n$. Методы выбора гиперпараметров разберём ниже.
2. Подбираем метрику расстояния $d(\cdot, \cdot)$. Чаще всего евклидово расстояние используется для числовых переменных, Хэмминга — для категориальных.

   ВАЖНО

   Если метрика того требует, необходимо масштабировать данные. Иначе признаки с большим диапазоном значений будут доминировать при вычислении расстояний.

3. Вычисляем расстояния от нового объекта $\vec{x}$ до каждого x_i из обучающей выборки:

$$\{d(\vec{x}, x_i) \mid i = 1, \ldots, n\}.$$

4. Ищем окрестность $N_k(\vec{x})$ — набор k пар (x_j, y_j) из объектов обучающей выборки, для которых расстояния $d(\vec{x}, x_j)$ оказались наименьшими среди всех вычисленных. Индекс j используется для обозначения индексов тех объектов, которые попали в число соседей, — здесь мы работаем с подмножеством из исходной выборки.

5. Предсказываем класс нового объекта.
   - **В задаче классификации** предсказанным классом $\hat{y}$ будет самый распространённый среди k соседей. Наибольшая частота встречаемости среди меток классов y_j соседей из окрестности $N_k(\vec{x})$ называется **majority vote**.

$$\hat{y} = \text{mode}\left(y_j \mid (x_j, y_j) \in N_k(\vec{x})\right) = \arg\max_{k \in K} \sum_{(x_j, y_j) \in N_k(\vec{x})} I(y_j = k).$$

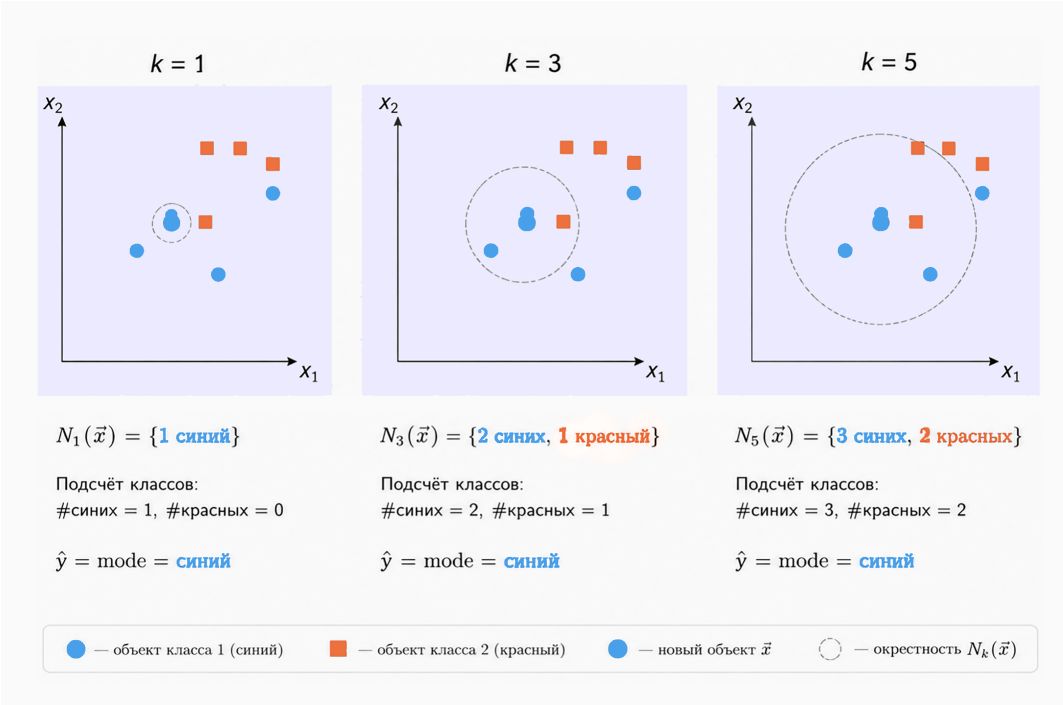


Рис. 12. Алгоритм kNN на примере классификации для количества соседей $k = 1$, $k = 3$, $k = 5$

   - **В задаче регрессии** предсказанным значением $\hat{y}$ будет **среднее значение (или, в одной из модификаций, медиана)** y_j соседей из окрестности $N_k(\vec{x})$:

$$\hat{y} = \text{mean}\left(y_j \mid (x_j, y_j) \in N_k(\vec{x})\right) = \frac{1}{k} \sum_{(x_j, y_j) \in N_k(\vec{x})} y_j.$$

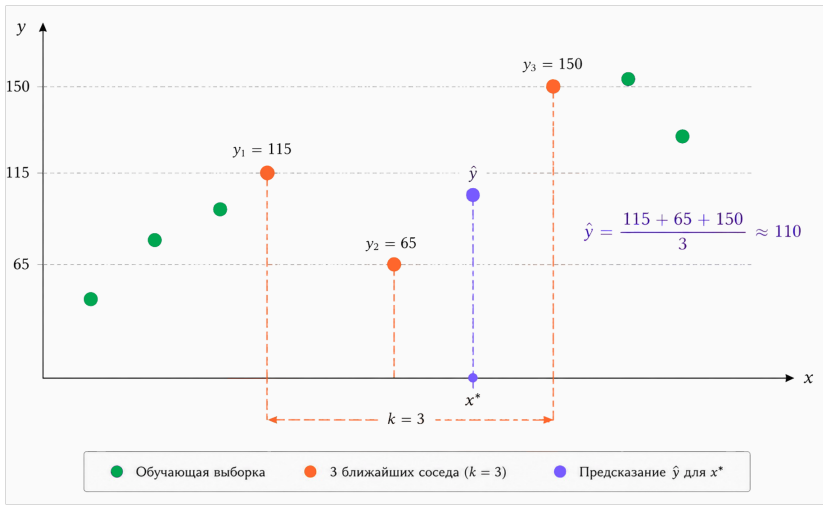


Рис. 13. Расчёт предсказания kNN для регрессии

На рисунках ниже показаны линии регрессии при количестве соседей, равном 1 и 3. Видно, что при $k = 1$ график проходит точно через точки выборки, при $k = 3$ аппроксимация сильно лучше, так как усредняется значение по трём точкам.

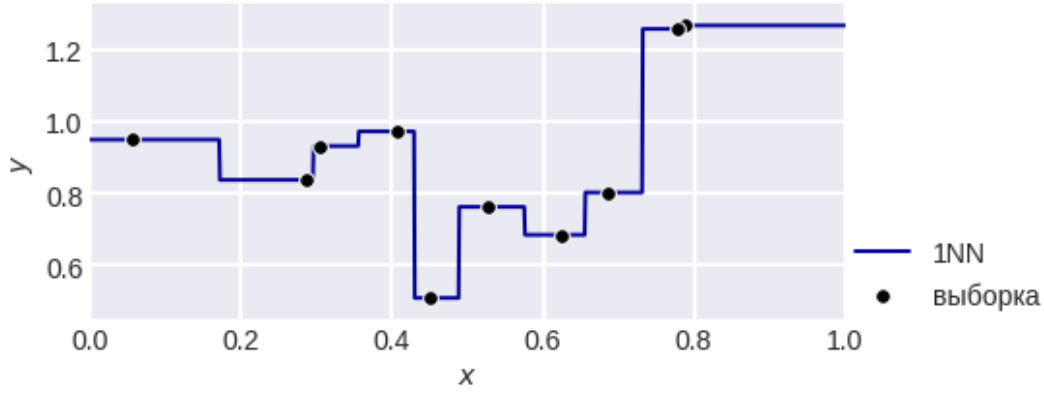


Рис. 14. Алгоритм kNN на примере регрессии для количества соседей $k = 1$. Источник

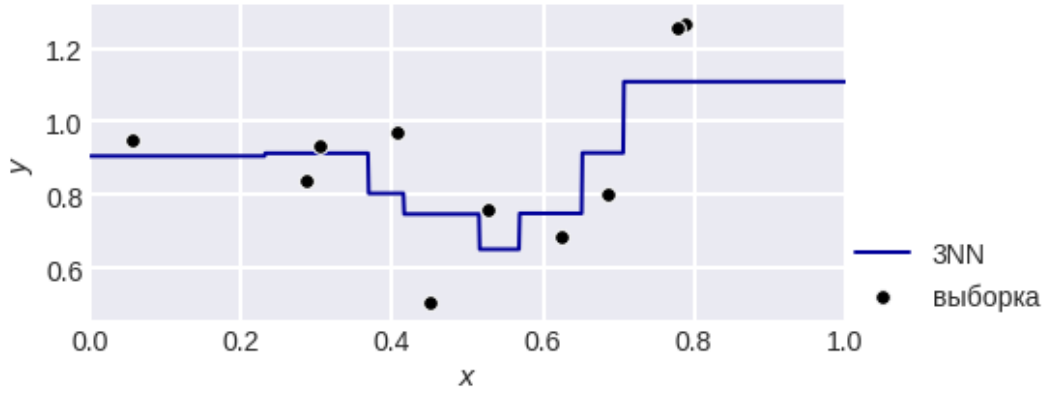


Рис. 15. Алгоритм kNN на примере регрессии для количества соседей $k = 3$. Источник

Интерактивную визуализацию работы алгоритма kNN для регрессии можно посмотреть на сайте mlweb.loria.fr.

ВАЖНО

В задачах классификации возможна ситуация равенства голосов по нескольким разным классам. В этом случае можно:

- изменить k (взять $k - 1$ и посмотреть, что получится);
- случайным образом выбрать один из классов, набравших максимальное число голосов.

3.2. КАК ВЫБРАТЬ ЗНАЧЕНИЕ k

Выбирать k нужно осторожно. Этот гиперпараметр определит, будет алгоритм локальным или обобщающим.

Если k маленький (например, $k = 1$):

- модель становится чувствительной к шуму и выбросам;
- поскольку модель подстраивается под каждый отдельный объект обучающей выборки, граница между классами становится сложной (извилистой);
- возникает риск переобучения: модель избыточно адаптируется под обучающие данные, из-за чего снижается её способность к обобщению.

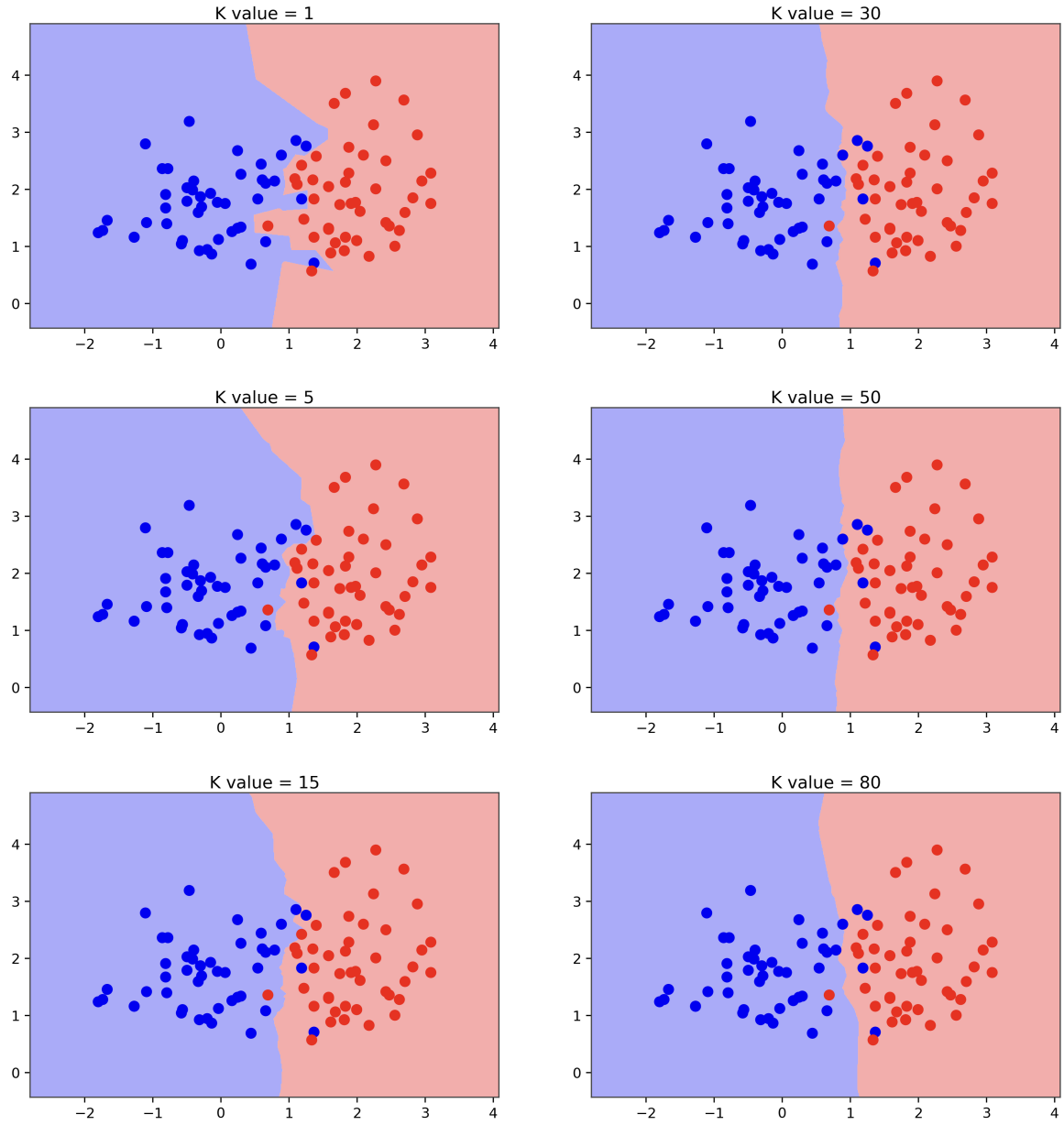


Рис. 16. Влияние гиперпараметра k на сложность границы решений

Если же k очень большой (например, $k = N$):

- модель становится более устойчивой к шуму за счёт усреднения влияния отдельных точек;
- граница решений становится очень гладкой, но теряется способность улавливать некоторые локальные закономерности данных;
- предсказание в любой точке пространства — это среднее значение по выборке.

Оптимальное значение k зависит от конкретной задачи и типа данных: заранее аналитически определить его нельзя. Стандартно используются методы контроля качества, например **кросс-валидация**. Как она работает:

- обучающая выборка разбивается на фолды;
- поочерёдно используется одна часть для валидации, остальные — для обучения модели с разными k (можно использовать `GridSearchCV` в `sklearn`);
- в результате выбирается такое k, которое даёт лучшую среднюю метрику на валидационных фолдах.

Кроме того, можно использовать **валидационную кривую** (*validation curve*).

- Для каждого значения k из некоторого диапазона мы строим модель для проверки.
- Оцениваем её качество на валидационной выборке.
- Строим график, где на оси X отложены значения k, а на оси Y — соответствующая им метрика качества.
- На графике выбираем такое значение k, при котором качество максимально. Если кривая выходит на плато (несколько k дают схожее высокое качество), выбираем наибольшее из них, потому что оно соответствует более простой модели.

Такой метод достаточно простой, интуитивный, быстрый, но менее надёжный и не гарантирует оптимальный результат. Однако его можно использовать как вспомогательный инструмент для визуализации процесса подбора гиперпараметров.

3.3. ВЗВЕШЕННЫЙ KNN

В стандартном алгоритме kNN все k соседей одинаково влияют на принятие окончательного решения. Интуитивно это не всегда правильно: логичнее предположить, что ближайшие соседи вносят больший вклад в предсказание, потому что, вероятно, больше похожи на новые объекты.

Отсюда возникает идея взвешенного kNN: в пределах выбранного k более близким соседям даётся больший вес (и, соответственно, влияние), а более далёким — меньший. Вес w каждого соседа j из окрестности $N_k(\vec{x})$ будет весовой функцией (в статистике она называется **ядром**, или **ядерной**; англ. *kernel function*) $K(\cdot)$ от расстояния $d(\vec{x},\, x_j)$:

$$w_j = K\Big(d(\vec{x},\, x_j)\Big).$$

Подобная функция $K(z)$, где z — произвольный действительный аргумент (скаляр), должна соответствовать следующим требованиям.

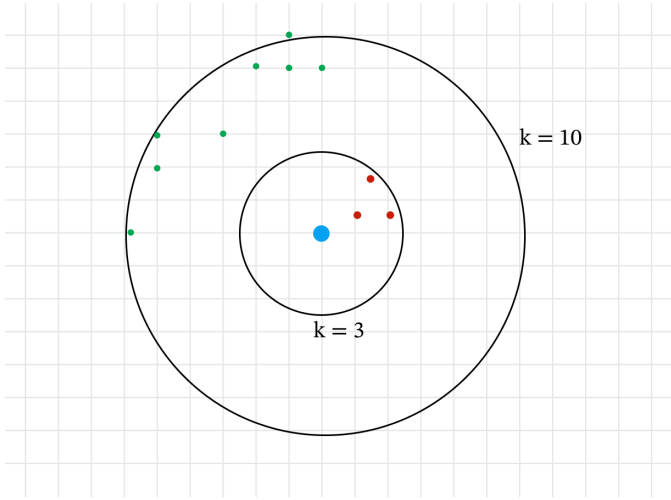


Рис. 17. Пример, показывающий, почему более близкие соседи могут быть важнее

1. $K(z) \geq 0$ для всех $z \in \mathbb{R}$ неотрицательно, как и расстояние.
2. $K(z) = K(-z)$ для всех $z \in \mathbb{R}$, то есть ядро симметрично относительно нуля.
3. $K(z)$ не возрастает при $z \to \pm\infty$, потому что мы отдаляемся от нашего объекта, а при $z = 0$ достигает максимума.
   По определению функция расстояния d всегда неотрицательна и не уходит в $-\infty$. Используется только «правая половина» симметричной функции ядра.

ВАЖНО

Не следует путать понятия ядра из статистики (функция взвешивания для скаляра) с ядрами из метода опорных векторов (SVM или функцией от двух векторов). Второй термин будет рассмотрен в следующих темах.

Выбор конкретной функции $K(\cdot)$ определяет, как именно расстояние преобразуется в вес. Часто используемые ядерные функции:

- обратное расстояние (*inversion kernel*):
$$\frac{1}{|z| + \epsilon};$$
- гауссово ядро (*Gaussian kernel*):
$$\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2} z^2\right);$$
- треугольное (*triangular*, *linear*):
$$(1 - |z|) \cdot I(|z| \leq 1);$$
- прямоугольное (*rectangular*, *top-hat*):
$$\frac{1}{2} \cdot I(|z| \leq 1);$$
- Епанечникова:
$$\frac{3}{4}(1 - z^2) \cdot I(|z| \leq 1);$$
- косинусное:
$$\frac{\pi}{4} \cos\left(\frac{\pi}{2} z\right) \cdot I(|z| \leq 1);$$
- квартическое (*quartic*):
$$\frac{15}{16}\left(1 - z^2\right)^2 \cdot I(|z| \leq 1).$$

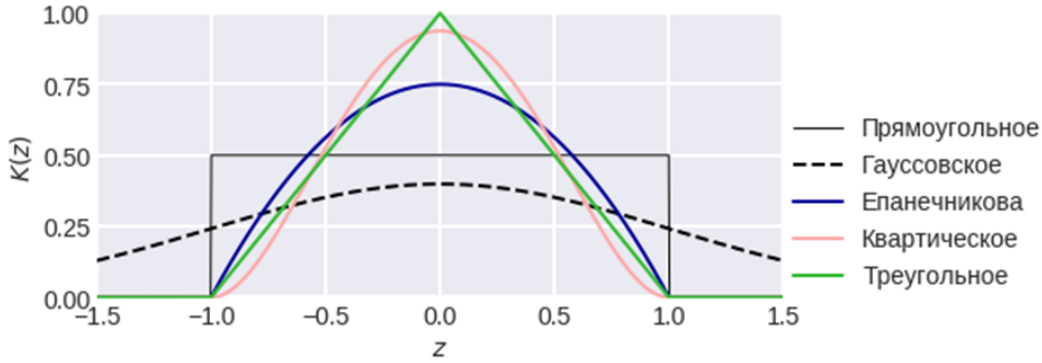


Рис. 18. Ядерные функции

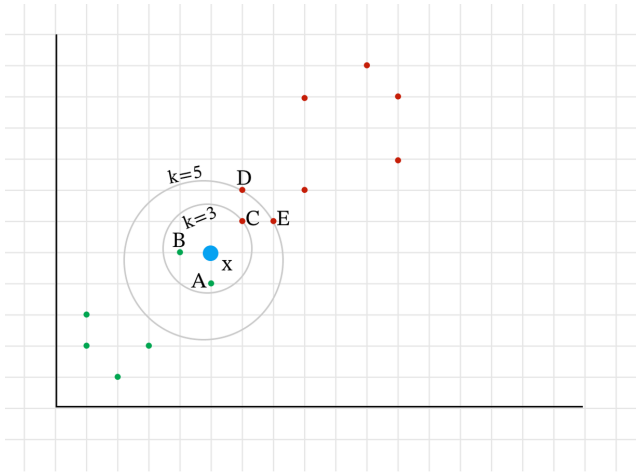
Таким образом, наши веса модифицируют шаг 5 алгоритма kNN.

- В задаче классификации получаем взвешенное голосование:

$$\hat{y} = \arg\max_{k \in K} \sum_{(x_j, y_j) \in N_k(\vec{x})} w_j \cdot I(y_j = k).$$

- В задаче регрессии — взвешенное среднее:

$$\hat{y} = \frac{\sum_{(x_j, y_j) \in N_k(\vec{x})} w_j y_j}{\sum_{(x_j, y_j) \in N_k(\vec{x})} w_j}.$$



$$k\big(d(x, x_i)\big) = \frac{1}{d(x, x_i)}$$

d_1:

$W_A = 1$

$W_B = 1$

$W_C = \frac{1}{2}$

$W_D = \frac{1}{3}$

$W_E = \frac{1}{3}$

k=3, ΣW=1+1=2, ΣW=0,5; 2>0,5 ⇒x∈●

k=5, ΣW=1+1=2, ΣW=1,17; 2>1,17 ⇒x∈●

d_2:

$W_A = 1$

$W_B = 1$

$W_C = \frac{1}{\sqrt{2}}$

$W_D = \frac{1}{\sqrt{5}}$

$W_E = \frac{1}{\sqrt{5}}$

k=3, ΣW=1+1=2, ΣW=0,7; 2>0,7 ⇒x∈●

k=5, ΣW=1+1=2, ΣW=1,6; 2>1,6 ⇒x∈●

Рис. 19. Пример работы взвешенного kNN для L_1- и L_2-метрик при разных k

Взвешенный kNN часто даёт более устойчивые результаты за счёт снижения влияния далёких соседей, попадание которых в окрестность может быть случайным. Такая модификация менее чувствительна к выбору k и позволяет избежать явной проблемы ничьей.

Как и другие алгоритмы, kNN обладает определёнными преимуществами и недостатками, которые необходимо учитывать при подборе модели для конкретной задачи.

4. Плюсы и минусы метода

Плюсы

- kNN прост в понимании и реализации.
- Метод непараметрический: мы не делаем жёстких предположений о форме распределения данных и функциональной зависимости.
- Он эффективно улавливает локальные паттерны, если схожие объекты находятся рядом в пространстве признаков. Алгоритм способен моделировать сложные границы решений.
- Демонстрирует хорошую работу «из коробки», например когда объекты естественным путём образуют хорошо разделяемые группы в пространстве признаков.
- Поскольку основная работа происходит на этапе предсказания, метод легко адаптируется к новым данным — нужно добавить их в обучающую выборку.

Минусы

- Высокая вычислительная стоимость на этапе предсказания: для каждого нового объекта необходимо вычислить расстояния для всех n объектов по m признакам обучающей выборки. Так мы получаем вычислительную сложность $O(n \cdot m)$ для вычисления расстояний и $O(N \cdot \log k)$ для отбора k ближайших соседей с помощью кучи при использовании самого эффективного алгоритма.
- В памяти нужно хранить всю обучающую выборку. Большие датасеты могут не поместиться в RAM.
- Метод очень чувствителен к выбору метрики и масштабированию признаков.
- Для каждой отдельной задачи нужно подбирать k, и это существенно влияет на результат. Просчёт кросс-валидации добавляет больше вычислительных затрат.

Отдельно стоит выделить проклятие размерности.

4.1. ПРОКЛЯТИЕ РАЗМЕРНОСТИ

ОПРЕДЕЛЕНИЕ

Проклятие размерности (англ. *curse of dimensionality*) — совокупность парадоксальных явлений, которые возникают при работе с данными в пространствах высокой размерности.

Теоретически это является большой проблемой для алгоритмов, подобных kNN.

Явление имеет несколько проявлений.

1. Неинформативные признаки добавляют шум в расчёт расстояния, искажая реальную близость между объектами.
2. Разница между расстоянием до самого дальнего ($d_{\max}$) и самого близкого ($d_{\min}$) соседа становится ничтожно малой по сравнению с величиной этих расстояний, то есть отношение

   $$\frac{d_{\max} - d_{\min}}{d_{\min}} \to 0.$$

   Это называется **концентрацией расстояний** (англ. *distance concentration*).
   Эффект имеет строгое математическое доказательство, которое выводится фундаментальной леммой из теории вероятностей: *случайная величина сходится к своему среднему, если её дисперсия стремится к нулю*. С её помощью также можно доказать закон больших чисел.

   ВАЖНО

   Материалы для более углублённого погружения в тему

   - Доказательство для расстояния Минковского:
     - bib.dbvis.de,
     - researchgate.net.
   - Расширение на другие метрики. В этой статье математически изящно показано, что многообразия неизбежно появляются в высокоразмерных данных. Однако это влечёт за собой новые сложности, связанные с нелинейной структурой и избыточностью признаков.
3. Рост количества признаков требует экспоненциального роста числа наблюдений для сохранения той же плотности данных. Если это условие не выполняется, например размер выборки фиксирован, то мы сталкиваемся с сильно разрежёнными данными в многомерном пространстве.

ВАЖНО

Несмотря на теоретические выводы, на практике kNN часто работает иначе.

Это объясняется гипотезой о многообразии.

ОПРЕДЕЛЕНИЕ

Гипотеза о многообразии (англ. *manifold hypothesis*) — это теория о том, что большинство реальных данных хоть и описаны в **высокоразмерном пространстве** (*ambient space*), но зачастую вполне компактно расположены на некоторой маломерной «поверхности» — **многообразии** (*manifold*).

kNN неплохо справляется с поиском *ближайших* соседей именно в этом многообразии, потому что расстояние (например, евклидово) в многомерном пространстве будет хорошей аппроксимацией расстояния в маломерном многообразии.

ПРИМЕР

Глобус — это объект в трёхмерном пространстве, точки на поверхности которого описываются широтой и долготой. Если измерять расстояние между городами евклидовой метрикой в 3D, то нужно учесть следующее.

- Придётся пройти сквозь саму толщу глобуса, корректнее — считать геодезическое расстояние (по двумерной поверхности глобуса — *маломерному многообразию*).
- В масштабе окрестности, где лежат *ближайшие* соседи, кривизна Земли не так сильно влияет на вычисления. Так, евклидово расстояние между Москвой и Зеленоградом примерно равно геодезическому.
- kNN способен находить осмысленных, правильных соседей, потому что данные часто компактно расположены на маломерном многообразии.

5. Регрессия Надарая–Ватсона

В предыдущей главе мы рассмотрели базовый kNN — простой, но мощный алгоритм, который:

- ищет k ближайших соседей для новой точки $\vec{x}$;
- предсказывает $\hat{y}$ как среднее (медиану) значений y_j соседей;
- взвешивает вклад каждого соседа.

Теперь рассмотрим эволюцию этой идеи для задачи регрессии. Почему бы не использовать все точки обучающей выборки, взвешивая вклад каждого наблюдения в зависимости от его удалённости от $\vec{x}$? При таком подходе:

1) далёкие точки практически не повлияют на результат из-за малого веса;
2) близкие точки внесут основной вклад.

Эта концепция лежит в основе регрессии Надарая–Ватсона.

ОПРЕДЕЛЕНИЕ

Регрессия Надарая–Ватсона — это непараметрический метод, который оценивает целевую функцию в точке x как *локальное* взвешенное среднее всех объектов из обучающей выборки. В ней веса определяются с помощью применения ядерной функции к расстоянию между точками x и x_i, нормированному на *ширину окна*.

Мы аппроксимируем данные ядерным сглаживанием и вводим гиперпараметр $h > 0$.

$$\hat{y}(x) = \left[\sum_{i=1}^{n} K\left(\frac{d(\vec{x},\, x_i)}{h}\right) y_i\right] \Bigg/ \left[\sum_{i=1}^{n} K\left(\frac{d(\vec{x}\,,\, x_i)}{h}\right)\right].$$

ОПРЕДЕЛЕНИЕ

Ширина окна (англ. *bandwidth*) — гиперпараметр h, отвечающий за радиус влияния ядра: в каком «окне» точка всё ещё считается достаточно близкой, чтобы получить значимый вес. Можно сказать, что h — это масштаб, в котором расстояния имеют значение.

В качестве иллюстрации алгоритма Надарая–Ватсона рассмотрим точку $x = 0{,}7$ на рисунке ниже.

Значение y в этой точке предсказывается не по одной формуле, а как средневзвешенное значение всех имеющихся наблюдений. Чем дальше точка данных от $x = 0{,}7$, тем меньше её вклад в итоговое предсказание (этот эффект отражён убывающей интенсивностью красного цвета на заднем плане)

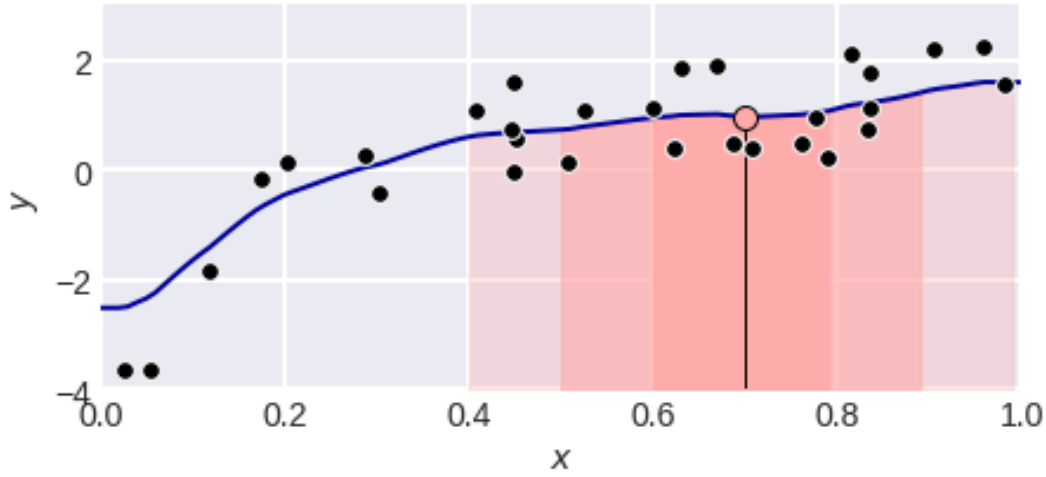


Рис. 20. Регрессия Надарая–Ватсона. Источник

ВАЖНО

Этот гиперпараметр — своеобразный аналог k в kNN, оптимальное значение которого подбирается с помощью кросс-валидации: при маленьком h значимый вес получат только самые близкие к $\vec{x}$ точки.

Таким образом, основное отличие от взвешенного kNN заключается в способе задания локальной окрестности. Во взвешенном kNN фиксируется число соседей k, тогда как в регрессии Надарая — Ватсона фиксируется ширина окна h, определяющая масштаб, на котором учитывается близость объектов.

При делении небольшого расстояния d на очень маленькое h получается большой аргумент функции ядра (чем он больше, тем меньше вес). Это приводит к быстрому затуханию влияния соседей с расстоянием. Полученная регрессионная кривая начинает «дёргаться», подстраиваясь под каждую точку обучающей выборки, что ведёт к переобучению (см. рис. 21).

Большое h усредняет по большой окрестности, потому что аргумент d/h растёт медленно. Это приводит к очень гладкой кривой и сильно упрощает аппроксимацию.

При $h \to \infty$ предсказание стремится к среднему по всем y_i.

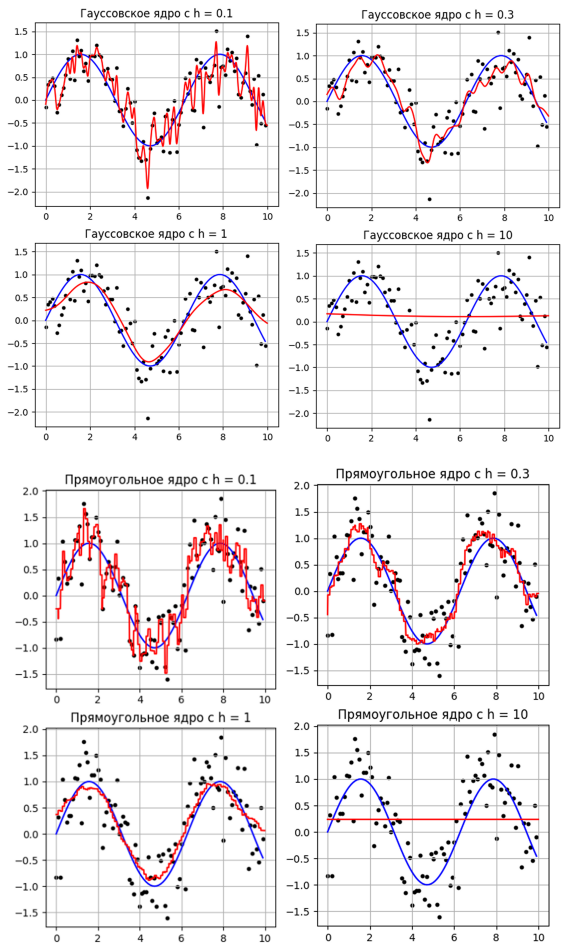


Рис. 21. Аппроксимация функции регрессией Надарая–Ватсона для гауссова и прямоугольного ядра при разной ширине окна h. Источник

ВАЖНО

Более подробную математическую постановку и визуализацию ядерного сглаживания локальных регрессионных методов (включая интерактивные примеры) можно найти на сайте egarpor.github.io.

5.1. ПЛЮСЫ И МИНУСЫ МЕТОДА

Плюсы

- Достаточно понятная идея локального взвешенного усреднения.
- Это непараметрическая модель, которая может улавливать сложные нелинейные связи.
- При использовании подходящего h и гладкого ядра (например, гауссова) даёт гладкие предсказания, что обеспечивает стабильность и предсказуемость прогноза.
- Хорошо адаптируется к локальным особенностям данных, потому что предсказание в каждой точке зависит от её окрестности.

Минусы

- Высокая вычислительная сложность: необходимо считать n расстояний, соответствующих весов, суммировать для каждого из них предсказания в новой точке, что медленно для больших n.
- Нужно хранить всю выборку.
- Можно столкнуться с проклятием размерности.
- Необходим подбор гиперпараметра h по аналогии с k в kNN.
- Модель чувствительна к масштабированию в зависимости от выбранной метрики расстояния.

6. Fast kNN

Главный недостаток kNN — его высокая вычислительная сложность. Однако существуют методы оптимизации и ускорения поиска ближайших соседей (*approximate nearest neighbors*).

ОПРЕДЕЛЕНИЕ

Поиск приближённых соседей (англ. *approximate nearest neighbors*, или *ANN*) — это методы приближённого поиска, которые жертвуют гарантированной точностью ради значительного выигрыша в скорости.

Вместо того чтобы находить *абсолютно* ближайших соседей, ANN находит кандидатов, которые с высокой вероятностью являются одними из ближайших.

Одна из идей: предварительная организация обучающей выборки в специальную структуру данных, которая позволит быстро отсечь большие группы нерелевантных точек.

К таким методам относятся следующие.

→ **KD-деревья** — древовидные структуры, в которых пространство признаков рекурсивно разбивается на две части по одному из признаков до того момента, пока в каждом листе (последнем результате разделения) не останется определённое количество точек.
Когда поступает новый объект, мы спускаемся в лист, в котором он мог бы находиться, если бы был в обучающей выборке. Затем возвращаемся обратно к корню дерева, заходя в ветви, которые не посещали.

- Если расстояние до гиперплоскости образуемой ветви больше любого из k найденных на данный момент расстояний, то всю ветвь можно отбросить.
- Если меньше — спуститься вглубь и повторить процесс.

В пространствах низкой и средней размерностей такие алгоритмы могут значительно ускорить поиск, но в высокой размерности работают со скоростью kNN. Вместо разбиения гиперплоскостями возможно разделение гиперсферами — так работают *ball trees*.

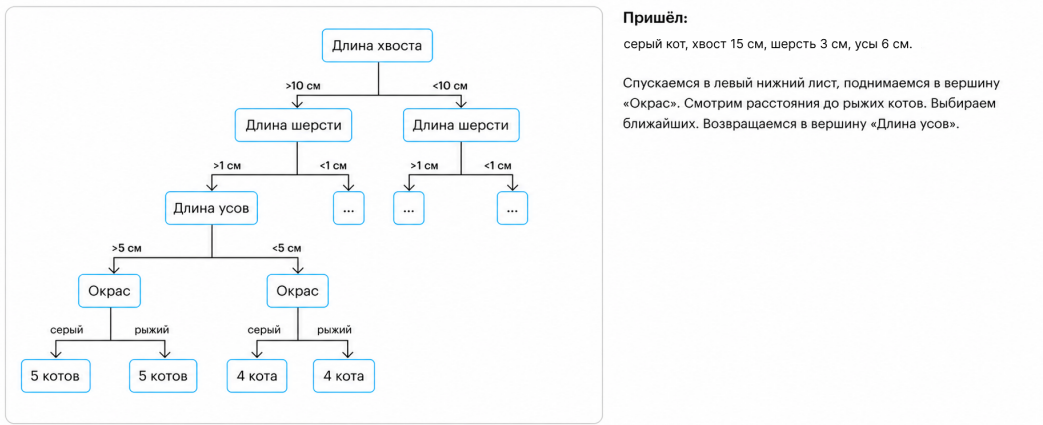


Рис. 22. Алгоритм KD-деревьев

→ **Locality-sensitive hashing (*LSH*)** — метод хеширования, при котором точки преобразуются в пространство меньшей размерности с сохранением свойства схожести. Похожие объекты с большой вероятностью получат одинаковые хеш-коды и попадут в одну корзину.

→ **Методы на основе графов** (например, *hierarchical navigable small worlds*, или *HNSW*) — подходы, которые позволяют обойти вершины разрежённого графа (где вершины — наблюдения, а рёбра — связи между близкими точками). Поиск начинается с разрежённого уровня, переходя на более плотные для точного определения соседей. Этот метод — один из наиболее эффективных.

→ **Data reduction / prototype selection** — методы уменьшения объёма данных путём замены некоторых групп точек их «представителями» или удаления избыточных точек.

→ **Annoy (*approximate nearest neighbors oh yeah*)** — открытая библиотека от Spotify, написанная на C++ (работает в Python). Она реализует построение некоторого количества деревьев с рандомными гиперпараметрами.

→ **FAISS*** **(*Facebook AI similarity search*)** — библиотека, специально разработанная для эффективного поиска векторной схожести в больших наборах данных.

Она предлагает:

- *exact search* (точный поиск): аналог обычного kNN (`IndexFlatL2`);
- *approximate search* (приближённый поиск):
  - кластеризация (`IndexIVFFlat`, `IndexIVFPQ`) — векторы делятся на кластеры, и поиск идёт только в нескольких ближайших;
  - графовые методы (`IndexHNSWFlat`) — строится граф близости векторов, поиск производится по рёбрам этого графа;
  - квантование (`IndexPQ`) — векторы сжимаются (квантуются) для экономии памяти.

Преимущества FAISS*:

- FAISS* значительно снижает вычислительную сложность поиска, особенно при использовании GPU, благодаря чему даже exact search работает быстрее;
- проблема хранения всех данных становится менее острой — можно использовать *product quantization* (PQ) для сжатия векторов, что требует меньше памяти, но остаётся необходимость хранения некоторого представления о данных;
- подбор оптимального k через кросс-валидацию ускоряется, потому что ускоряется поиск на каждой итерации.

FAISS* оптимизирована для работы с высокоразмерными пространствами, что делает её универсальным инструментом для задач, где требуется эффективный поиск и анализ данных в векторном представлении.

ВАЖНО

Материалы для более углублённого погружения в тему:

- по Annoy:
  - статья о ближайших соседях и векторных моделях;
  - GitHub: Spotify;
- по FAISS*:
  - GitHub: FAISS*;
  - подробное описание алгоритмов.
  - описание GPU-имплементации;
  - статья от авторов библиотеки.

* FAISS принадлежит компании Meta, признанной экстремистской и запрещённой на территории Российской Федерации.

7. Если кратко

- → Метрические алгоритмы — методы машинного обучения, которые делают предсказание для объекта на основе сравнения с соседями. Они непараметрические.
- → Расстояние — главный инструмент метрических алгоритмов.
- → Предсказание для объекта делается на основе свойств его соседей.
- → Идея соседства обобщается на разные типы задач. В задаче классификации предсказанным классом будет самый часто встречающийся класс среди соседей (*majority vote*). В задаче регрессии — среднее (или медианное) значение соседей.
- → Гиперпараметр k определяет, будет алгоритм локальным или обобщающим.
- → Метрика подбирается в зависимости от задачи и типа данных.
- → У метрических алгоритмов высокая вычислительная сложность, но существуют быстрые и эффективные реализации для поиска соседей.